AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Branching and boundary tests

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Write the rule. Follow the first match. Test equality.

Trace if/elif/else without confusing skipped checks and false checks.

Keep one habit: Duration chain → book-count validity order → boundary/mutation cases → selected lamp trace → portfolio evidence.

MAIN LESSON · SECTION 1

Which branch runs, and which conditions are skipped?

> Trace the first matching branch and test the exact decision boundary.

Your learning goals

I can trace the selected **branch** in an if elif else chain.

I can use **indentation** to identify which statements belong to a **branch**.

I can test values at and around a decision **boundary**.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Boolean **condition**s. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Trace the selected **branch** in an if elif else chain
- Use **indentation** to identify which statements belong to a **branch**
- Test values at and around a decision **boundary**
- Does minutes <= 12 accept exactly 12? How does < differ?

My starting explanation

Your end goal

Trace **branch**ing and **boundary** tests and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Write the duration rule before the branches

> Negative is Invalid, zero through twelve is Short, and larger integers are Long.

The complete classification contract

A fictional activity sorter labels a supplied integer duration. Negative values are Invalid; values from zero through 12 are Short; larger values are Long. The labels describe duration only.

Write this requirement before the program so that a test has an expected answer independent of the code.

Read colons, alignment and indentation

An if statement selects a block when its **condition** is true. An elif is considered only if earlier **condition**s in the same chain were false. An else handles the remaining cases. Colons end the **condition** lines.

Use four spaces for the indented statements and keep related **branch**es aligned.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Trace checked and skipped conditions separately

print runs after the chain because it is outside every branch.

Connect input to selected path

One chain chooses one label

INPUT	CHECKS IN ORDER	SELECTED LABEL
-1	Below zero is True	Invalid
12	Below zero False; <= 12 True	Short
13	Both checks False	Long

Later branches in this chain are skipped after a condition succeeds.

Complete duration-classifier script

```
minutes = 12
if minutes < 0:
    label = "Invalid"
elif minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

Inspect tested and skipped checks

minutes	minutes<0	minutes<=12 in this chain	Selected label
-1	True	Skipped	Invalid
12	False	True	Short
13	False	False	Long via else

Read assignment and output scope

The **output** is Short. For 12, the first **condition** is false and the second is true. The else block is skipped. print(label) is not indented, so it runs after the selected **branch**.

Each path assigns label before this line reads it. Change only minutes and save before each fresh run.

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare a genuine branch chain with repeated independent IFs

> The second independent decision can overwrite the first label.

Predict the supported change

- For -1, 12 and 13, record checked **condition**s, skipped **condition**s and the final label. Run each as a fresh script after predicting.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured variant

- Change: Remove the first negative-count **branch**.

▼ Check after predicting

Prediction: A count of -2 would incorrectly receive small.

Reason: Negative two is less than five, but it is not a permitted measured count.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace negative, below-five and equality book counts

> Validity is checked before the small/large decision.

Read the complete pictured rules and scope

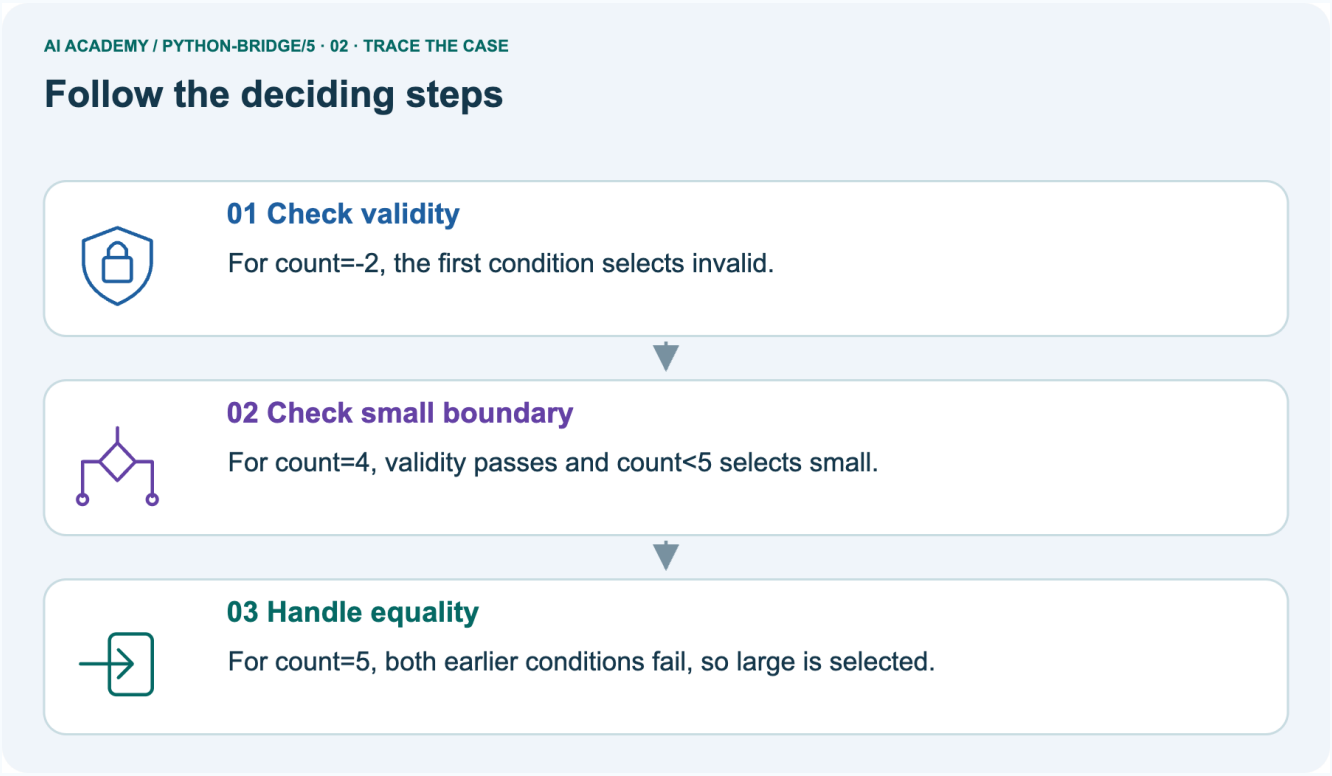
Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

Keep three toy contracts distinct. Activity durations use Invalid/Short/Long with upper Short boundary 12. Pictured book counts use invalid/small/large with small strictly below 5. The independent lamp-reading task states its own 0..100 range and 30 threshold. All these examples use supplied integer inputs; numeric comparisons do not implement arbitrary type validation or physical device control.

The abbreviated picture notation is expanded into runnable indented Python below.

Trace validity and the strict boundary



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Check validity	For count=-2, the first condition selects invalid.
2. Check small boundary	For count=4, validity passes and count<5 selects small.
3. Handle equality	For count=5, both earlier conditions fail, so large is selected.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

An if/elif/else chain selects the first matching **branch**. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Complete indented implementation

```
count = -2
if count < 0:
    label = "invalid"
elif count < 5:
    label = "small"
else:
    label = "large"
print(label)
```

Predict removing negative-count handling

- Change one thing: Remove the first negative-count **branch**. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting

AI ACADEMY / PYTHON-BRIDGE/5 · 04 · COMPARE THE CASES

Change one thing. Trace what follows.

Original case

For count=5, both earlier conditions fail, so large is selected.

Changed condition

Remove the first negative-count branch.

Worked changed-case result

A count of -2 would incorrectly receive small.
Why: Negative two is less than five, but it is not a permitted measured count.

Follow the labelled picture with the matching explanation. Open the vector diagram

State the numeric-input limitation

- The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

- Read the labelled picture: Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

▼ Separate duration tests · Why equality matters

Input minutes	Expected label	Reason for this test
-1	Invalid	Negative input rejection
0	Short	Lowest valid duration
11	Short	Just below the upper boundary
12	Short	Exact upper boundary
13	Long	Just above the boundary

These tests exercise every **branch** and the equality case. A change from <= 12 to < 12 is exposed by 12; testing 11 alone misses it.

Branch coverage means every **branch** was exercised, but it does not prove every **condition** is correct.

> **Suppose the program uses < 12 instead of <= 12. What is printed for 12, why does that violate the written rule, and which earlier test should still pass after repair?**

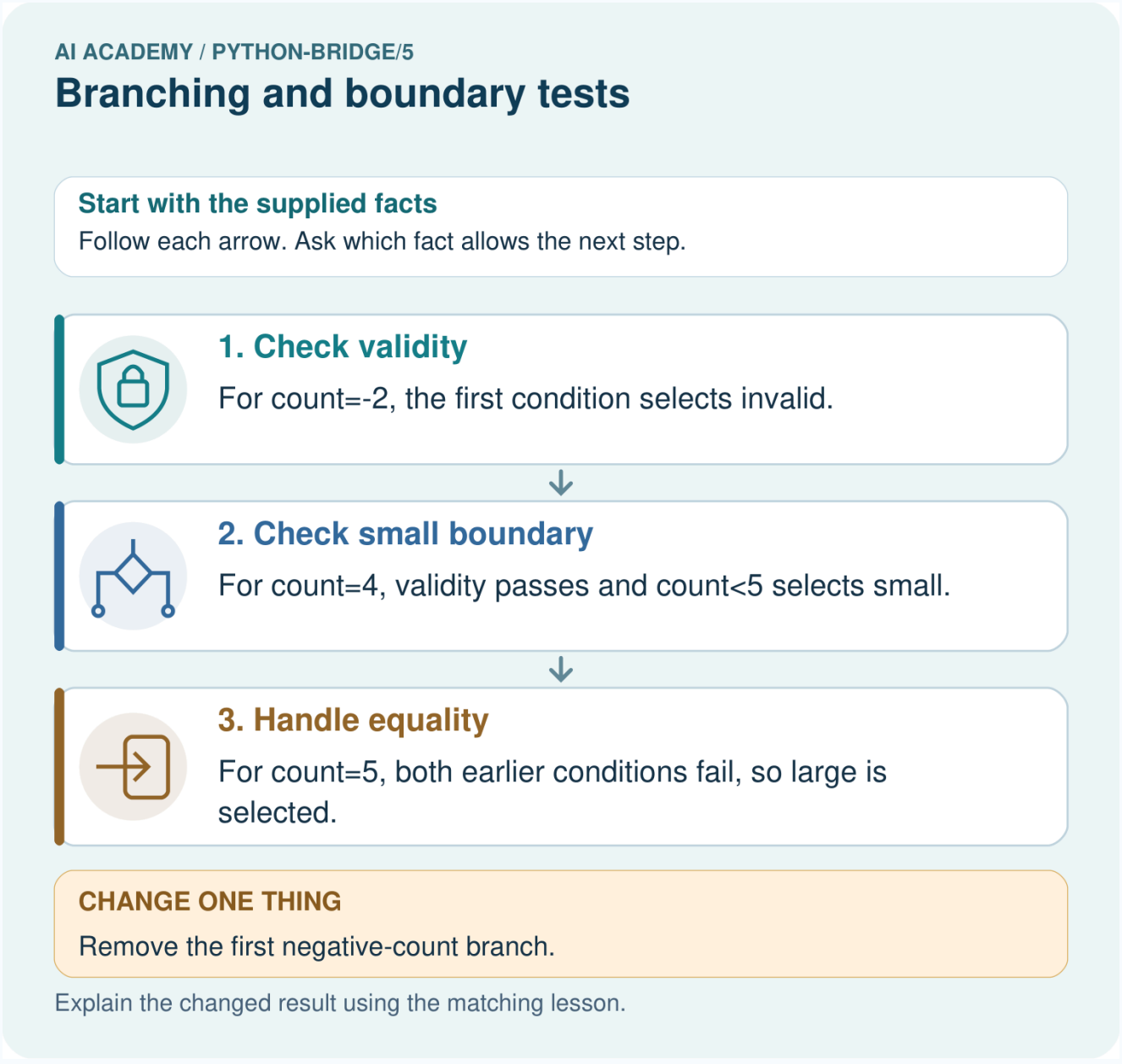
▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Branching and boundary tests

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find the independent IF that overwrites Invalid

> The else belongs to its own associated if, not an earlier separate decision.

Read the complete faulty example

```
minutes = -1
if minutes < 0:
    label = "Invalid"
if minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

Trace the overwritten label and indentation effect

This faulty version prints Short. The second if starts a separate decision and overwrites Invalid. Its else belongs to that second if. Restoring elif makes the second test depend on the first test failing.

Moving print inside one **branch** can also hide **output** on other paths; **indentation** affects behavior, not just appearance.

State input/measurement limits

The script rejects negative integers, but it does not validate arbitrary types or check a measured duration. A supplied text value can raise an error during comparison.

Keep the stated integer **input** contract and report the limitation honestly.

Make the smallest chain repair

- Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise path tracing with less help

> Do not mark an unexecuted condition False; label it skipped.

Explain the procedure

- Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented **branch**es.

Suppose the program uses < 12 instead of <= 12. What is printed for 12, why does that violate the written rule, and which earlier test should still pass after repair?

Explain your prediction to a partner before comparing results.

Complete checked/skipped W2

Use workbook W2 for the connected example “**Branch**ing and **boundary** tests”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Audit every-condition-runs reasoning

- Use the worked case for **Branch**ing and **boundary** tests. Proposed explanation: “Every **condition** in the chain always runs”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured validity order

- Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

My explanation using a pictured step and the given facts

Reserve independent work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace the tested path

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Check six complete branch and mutation cases

> A useful test distinguishes the intended rule from a plausible wrong one.

All supplied inputs and expected paths

Complete duration case	Expected label	Why keep it
-1	Invalid	Negative branch, later elif skipped

Complete duration case	Expected label	Why keep it
0	Short	Lowest valid count
11	Short	Just below inclusive upper limit
12	Short	Equality distinguishes <= from <
13	Long	Just above the upper boundary
Faulty two-IFscript at -1	Short, which violates the written rule	The second if overwrites Invalid

Predict, run or trace, repair and retest

1. Use the complete duration-classifier script for the five normal/boundary inputs. Predict checked conditions, skipped alternatives, selected label and final print. Then compare the supplied two-independent-if version at -1and repair only the second if to elif. A second intentional mutation changes <=12 to <12: test 12 to expose it, then rerun 11 and 13 to check neighboring behavior.

   Cases -1, 2, 20 can visit all branches while missing that equality bug, so branch coverage is not proof that every condition is correct. Preserve wrong outputs as observed evidence rather than silently replacing them. Keep the independent lamp-reading script unused until the individual attempt.

Complete original and faulty scripts

```
minutes = 12
if minutes < 0:
    label = "Invalid"
elif minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

```
minutes = -1
if minutes < 0:
    label = "Invalid"
if minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

Keep checked/skipped and output records

For -1, 12 and 13, record checked **condition**s, skipped **condition**s and the final label. Run each as a fresh script after predicting.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the selected lamp-reading chain

> Validity and operational threshold are separate checks.

Selected complete task and scope

Predict the action for 30, then trace 29 and -1. Explain why validity is checked before the lamp decision.

```
reading = 30
if not 0 <= reading <= 100:
    action = "invalid"
elif reading < 30:
    action = "on"
else:
    action = "off"
print(action)
```

Use this selected script's own supplied integer reading and numeric validity range. action is only a stored label, not a physical command. Trace the exact boundary and invalid input before considering the on/off branch; do not import the duration or book thresholds.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier branch transfer

Use an integer tiles count. Label a negative value Invalid, zero Empty, 1 through 6 Small and values above 6 Large. Write an ordered if elif else script and predict -1, 0, 1, 6 and 7. Include one print after the chain.

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Trace branch order

Attempt this without hints

Use feedback to improve

MAIN LESSON · SECTION 11

Keep the requirement, code and test evidence together

> A mutation should change one condition so the revealing test is clear.

This week's contribution

Add a labelled **branch**ing and **boundary** tests **evidence** sheet to Python readiness portfolio. Show how you can trace the selected **branch** in an if elif else chain.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain the first matching branch and the skipped path

> A correct trace needs the written rule and exact boundary.

Explain without notes

- Close your notes. Explain this goal in your own words: Trace the selected **branch** in an if elif else chain

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Branch	A possible path through a program
Condition	An expression used to decide which path to take
Indentation	Leading spaces that identify a Python code block
Boundary	A value where a classification may change
Regression check	A previous test rerun after an edit to detect a new failure

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented **branch**es.
- Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.
- A repair changes the small-count **condition** from < 6 to <= 6. Name the case that exposes the original error and two different kinds of earlier cases to rerun.

Explain which evidence supports your conclusion and which claim remains unproved.

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- For each run, record **input**, expected label, **condition**s actually checked and observed label. Mark skipped **condition**s as skipped rather than pretending the program evaluated them.

  After a repair, rerun earlier cases as well as the failing one. Week 6 will repeat a block and track changes across iterations.

- A repair changes the small-count **condition** from < 6 to <= 6. Name the case that exposes the original error and two different kinds of earlier cases to rerun.

> **A learner tests -1, 2 and 20 and concludes the sorter is completely correct because all three branches ran. Explain what important case is missing and why branch coverage alone is insufficient.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

One chain chooses one label

INPUT	CHECKS IN ORDER	SELECTED LABEL
-1	Below zero is True	Invalid
12	Below zero False; <= 12 True	Short
13	Both checks False	Long

Later branches in this chain are skipped after a condition succeeds.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Trace the selected branch in an if elif else chain	Explain the deciding step, show a trace or test, and state one limit.
Use indentation to identify which statements belong to a branch	Explain the deciding step, show a trace or test, and state one limit.
Test values at and around a decision boundary	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.
- Which labelled step matches this detail? For count=5, both earlier **condition**s fail, so large is selected.
- 1. Check validity
- 2. Handle equality
- 3. Check small **boundary**
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> ---
>
> ---
>
> ---
>
> ---

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Handle equality For count=5, both earlier **condition**s fail, so large is selected. An if/elif/else chain selects the first matching **branch**.

Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Changed-case question and hint

- Changed-case practice: Remove the first negative-count **branch**. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. A count of -2 would incorrectly receive small.
- 3. We can decide the result without examining the changed **input** or the procedure.
- Hint if needed: Use the changed **condition** and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: A count of -2 would incorrectly receive small. Negative two is less than five, but it is not a permitted measured count.

Repair and keep the record

Review Boolean **condition**s (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

Review Values variables and types (python-bridge/2). Goal: Distinguish integer, floating-point, text and Boolean values. Short practice: Predict both print **output**s and explain why they differ.

Does count+tag work, and what happens to each valid **output** after count becomes ten?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

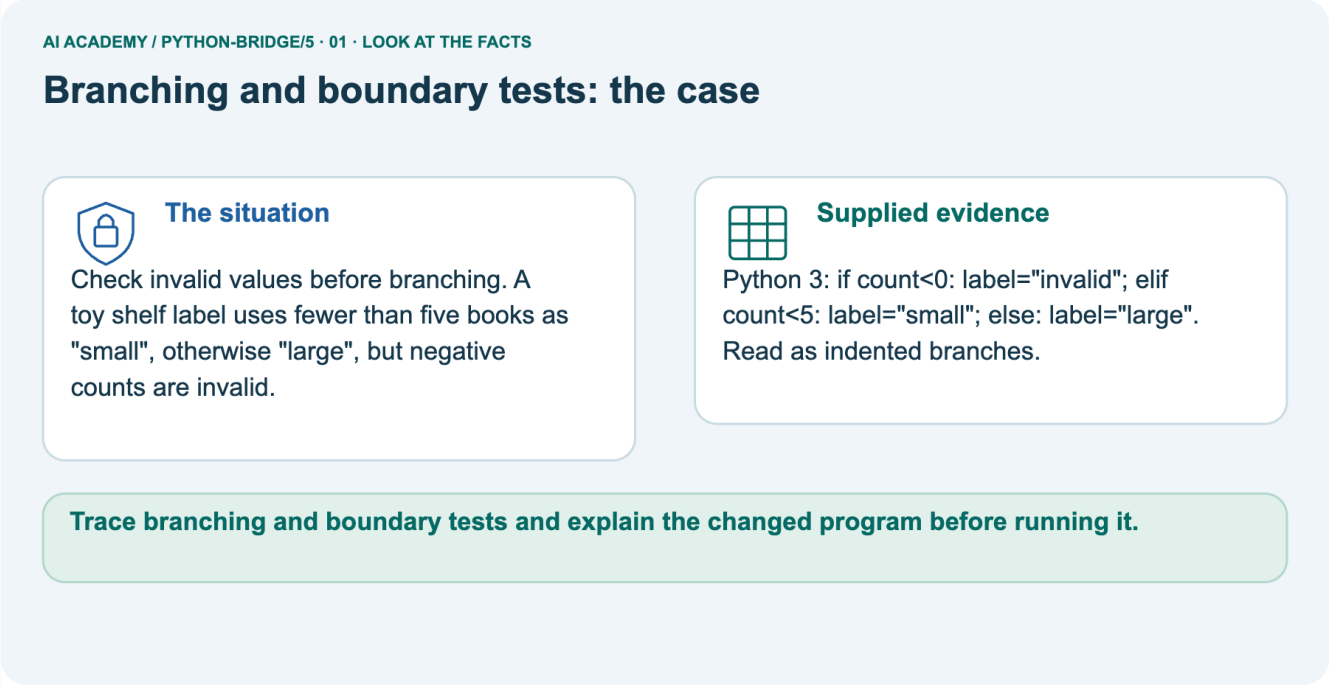
▼ **Reference · The six-picture journey**

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Branching and boundary tests: the case



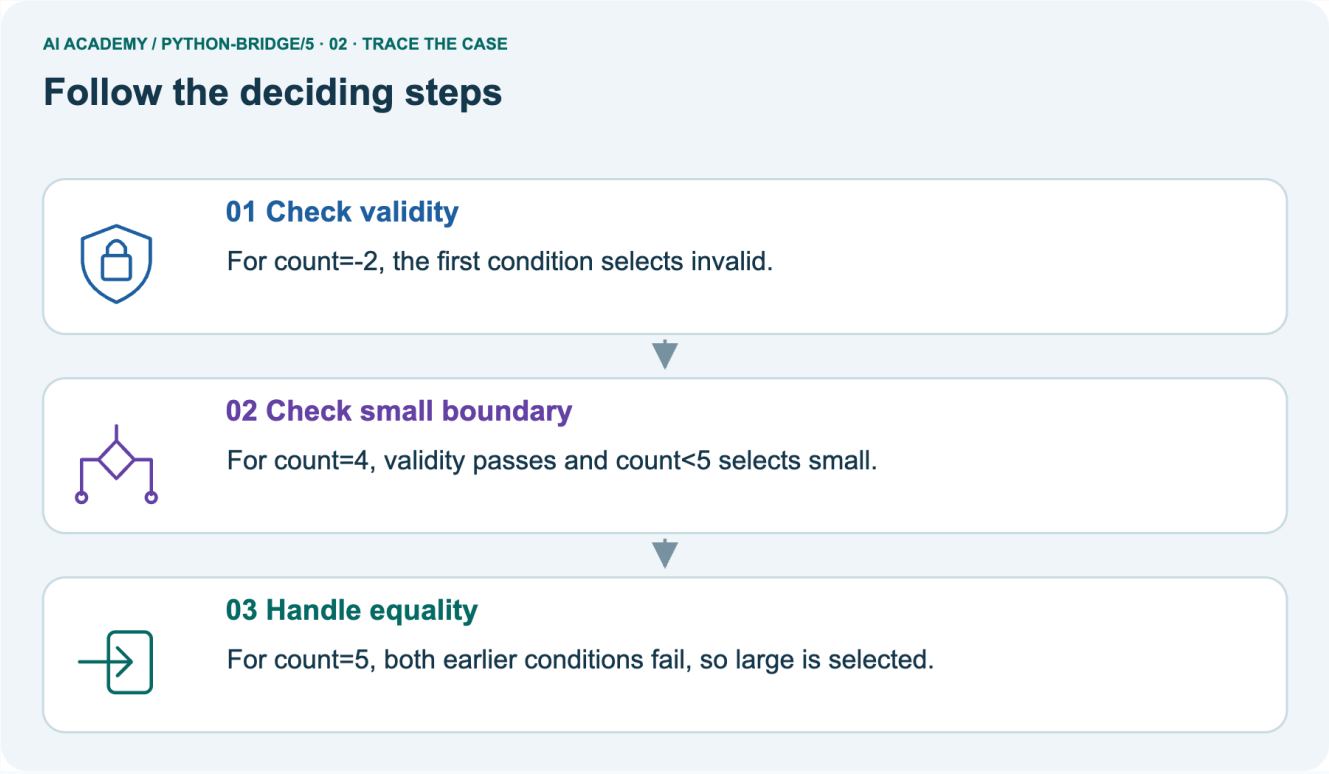
Follow the labelled picture with the matching explanation. Open the vector diagram

Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Supplied facts: Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

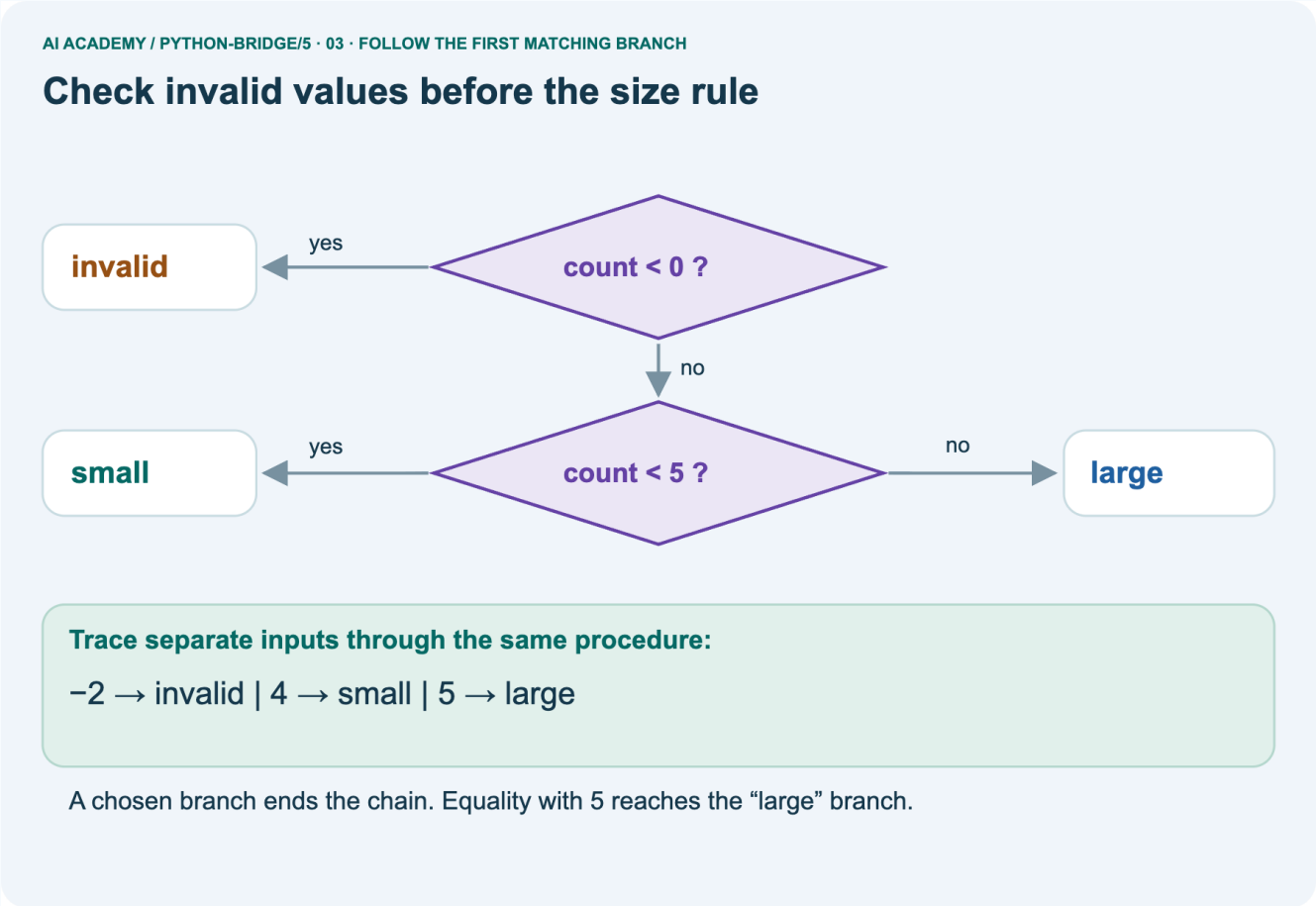


Follow the labelled picture with the matching explanation. Open the vector diagram

Check validity: For count=-2, the first **condition** selects invalid. → Check small **boundary**: For count=4, validity passes and count<5 selects small. → Handle equality: For count=5, both earlier **condition**s fail, so large is selected.

Point to each numbered stage and explain which recorded **input** it uses.

03. Check invalid values before the size rule

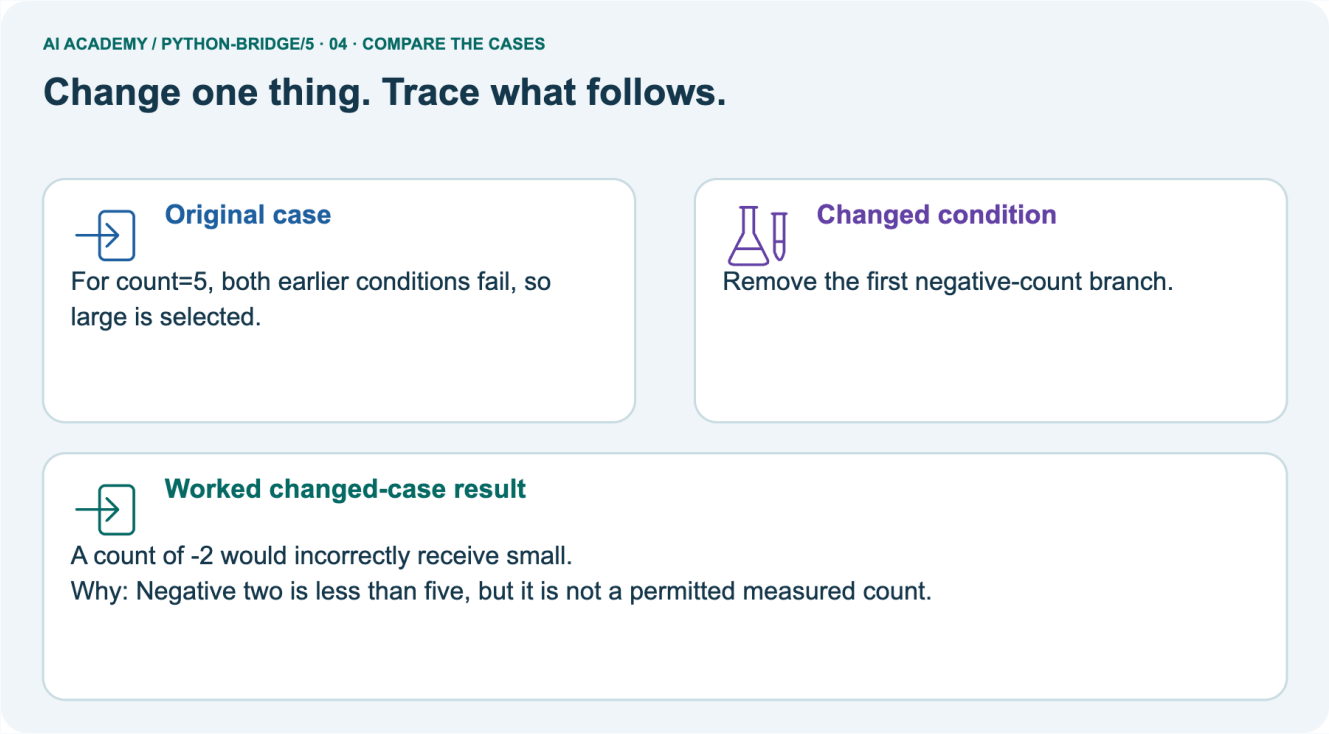


Follow the labelled picture with the matching explanation. Open the vector diagram

First ask count less than zero: yes selects invalid. Otherwise ask count less than five: yes selects small; no selects large. Separate **input** traces are minus two invalid, four small, five large.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



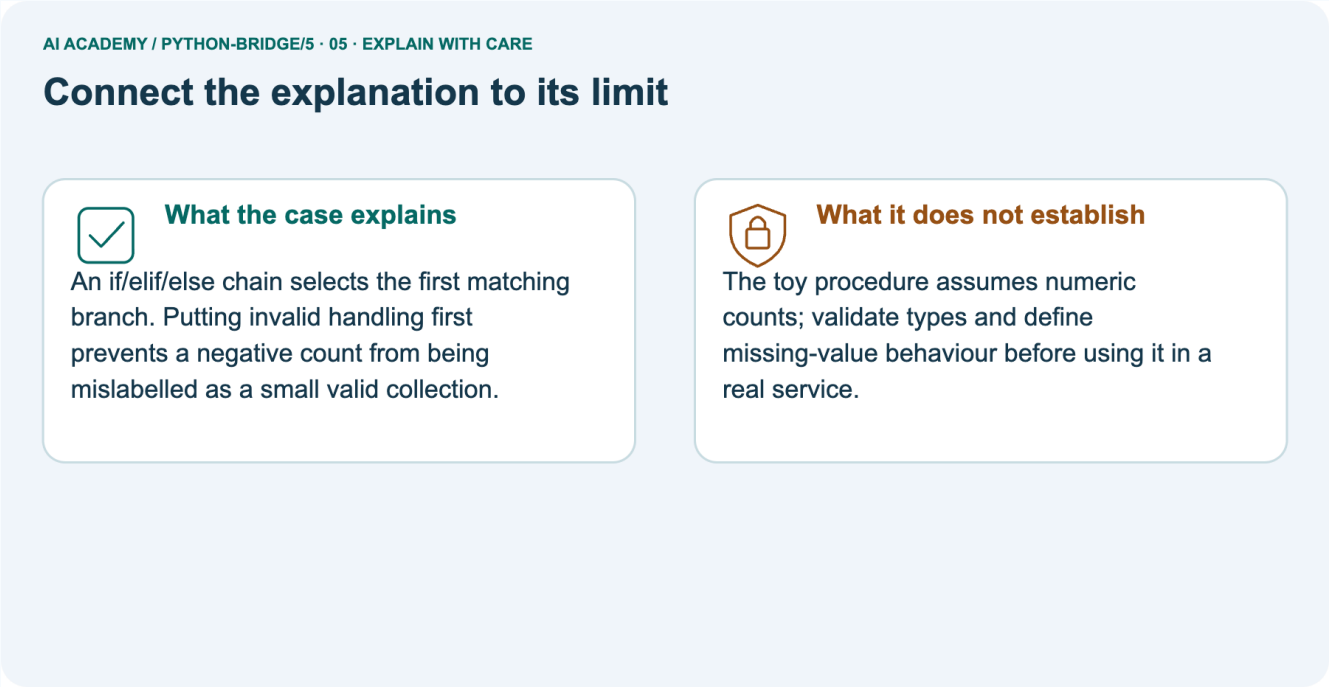
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: For count=5, both earlier **condition**s fail, so large is selected. Changed **condition**: Remove the first negative-count **branch**. Expected result: A count of -2 would incorrectly receive small.

Reason: Negative two is less than five, but it is not a permitted measured count.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



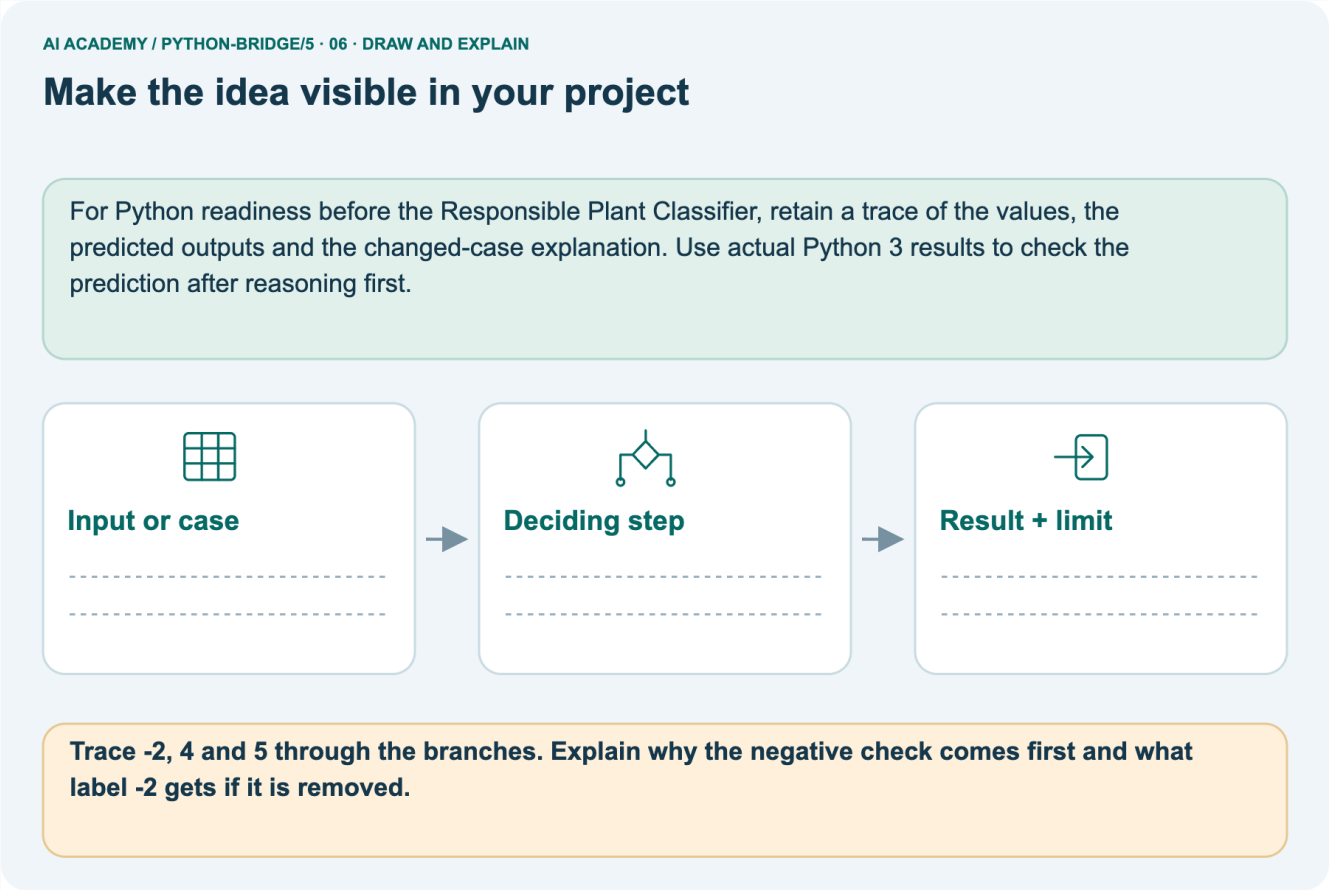
Follow the labelled picture with the matching explanation. Open the vector diagram

An if/elif/else chain selects the first matching **branch**. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Limit: The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Branching and boundary tests: the case

02. Follow the deciding steps

03. Check invalid values before the size rule

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.